NelimaL
Posted on Apr 22

Edit Manage Stats

SQL Without Confusion: Subqueries vs CTEs (Visual + Practical Guide)

#sql #dataanalytics #database #beginners

Understanding subqueries and CTEs (Common Table Expressions) becomes much easier when you can see how data flows. This guide uses simple diagrams and real-style datasets.

What is a Subquery?
A subquery is a query inside another query.

Visual Representation
Outer Query
↓
[Uses result from]
↓
(Subquery)

Example: Students scoring above average

```
SELECT first_name, marks
FROM students s
JOIN results r ON s.student_id = r.student_id
WHERE marks > (
    SELECT AVG(marks) FROM results
);
```

**How it Works
Step 1: Subquery runs
SELECT AVG(marks) → returns 81.6

Step 2: Outer query runs
Returns students with marks > 81.6**

Types of Subqueries

1. Scalar Subquery (Single Value)

Returns exactly one value (one row and one column). It is commonly used in SELECT lists or in WHERE clauses with simple comparison operators like = or >.

```
SELECT first_name
FROM students
WHERE student_id = (SELECT student_id FROM results WHERE marks = 90);
```

2. Correlated Subquery (Row-by-Row Execution)

Depends on the outer query and executes once for each row in the outer query
FOR each row in outer query:run subquery again

```
SELECT s.first_name
FROM students s
WHERE marks > (
    SELECT AVG(marks)
    FROM results r
    WHERE s.student_id = r.student_id
);
```

This can be slow because it runs multiple times.

3. Multi-row Subquery

Returns one or more rows. These usually require set operators such as IN, ANY, or ALL in the outer query.

```
SELECT first_name
FROM students
WHERE city IN (
    SELECT city FROM patients
);
```

This finds students living in the same cities as patients.

What is a CTE?

A CTE is a temporary result set defined using the WITH clause. It improves readability

and allows you to reuse query logic.

Basic Syntax:
WITH cte_name AS (
SELECT column1, column2
FROM table
)
SELECT * FROM cte_name;

Types of CTEs

1. Simple CTE-Used to simplify complex queries

```
WITH avg_marks AS (SELECT AVG(marks) AS avg_score FROM results)
SELECT s.first_name, r.marks
FROM students s
JOIN results r ON s.student_id = r.student_id
JOIN avg_marks a ON 1=1
WHERE r.marks > a.avg_score;
```

Flow
Step 1: avg_marks created
Step 2: Main query uses it

1. Multiple CTEs-Define more than one CTE in a single query

```
WITH student_marks AS (
    SELECT s.first_name, r.marks
    FROM students s
    JOIN results r ON s.student_id = r.student_id
),
top_students AS (SELECT * FROM student_marks WHERE marks > 80)
SELECT * FROM top_students;
```

1. Recursive CTE (Hierarchy) A CTE that references itself to repeatedly execute a query until a specific condition is met. It consists of an anchor member (initial result) and a recursive member (iterative logic). Example: Employee hierarchy (Hospital)

```
WITH RECURSIVE staff_hierarchy AS (
    SELECT patient_id, full_name
    FROM patients
```

```
    WHERE patient_id = 201
    UNION ALL
    SELECT p.patient_id, p.full_name
    FROM patients p
    JOIN staff_hierarchy sh ON p.patient_id = sh.patient_id)
SELECT * FROM staff_hierarchy;
```

Subqueries vs CTEs

Comparison Table

📊 Comparison Table

Feature	Subqueries	CTEs
Readability	Can become hard to read when nested	Much cleaner and easier to follow
Reusability	Cannot be reused easily	Can be referenced multiple times
Performance	Faster for simple queries	Better for complex logic
Execution	Often executed multiple times	Executed once (in many cases)
Debugging	Harder to debug	Easier to test step-by-step
Recursion	Not supported	Supported

When to Use What

Use Subqueries When:

- The logic is simple and short
- You only need the result once
- You want a quick inline filter Use CTEs When:
- The query is complex or layered
- You need to reuse logic
- You want cleaner, more maintainable SQL
- You are working with recursive data

Final Thoughts

Both subqueries and CTEs are essential tools in SQL. While subqueries are great for quick, simple tasks, CTEs shine when dealing with complex logic and improving readability.

As a data analyst, knowing when to use each can significantly improve both your query performance and the clarity of your code.

Top comments (0)

Code of Conduct • Report abuse

NelimaL

Detail-oriented Data Analyst with a foundation in Actuarial Science and a professional certification in Monitoring & Evaluation. Skilled in analyzing complex datasets to produce actionable insights.

JOINED
Mar 13, 2026

More from NelimaL

Mastering SQL Joins: A Practical Guide for beginners.
#sql #datascience

Understanding SQL Basics: DDL, DML, Filtering, and Data Transformation
#sql #database